GENESIS 2.5: optimisation and opt-in OpenCL/CUDA acceleration for the GENESIS/PGENESIS compartmental neural simulator

Karol Chlasta[a,*], Grzegorz Marcin Wójcik[b]

[a]*Kozminski University, ul. Jagiellońska 57/59, 03-301 Warsaw, Poland; WarsawIQ, Al. Jana Pawła II 43A/37B, 01-001 Warsaw, Poland*
[b]*Maria Curie-Skłodowska University, ul. Akademicka 9, 20-033 Lublin, Poland*

Abstract

GENESIS is a long-established compartmental neural simulator. GENESIS 2.5 adds optional CUDA and OpenCL backends to its `hsolve` solver, including a GPU Hines elimination for dendritic-tree populations, without changing existing models, scripts or MPI workflows. On 160 000 compartments over 10 000 steps an A100 runs 24.0× faster end to end than the CPU solver; in double precision it reproduces the CPU solver to every printed digit. Networks that exchange spikes every step gain nothing on the GPU, and we show why. The release also makes model construction linear and fixes two defects in the Hines solver of GENESIS 2.4.

Keywords: GENESIS, PGENESIS, compartmental neural simulation, Hines solver, GPU, CUDA, OpenCL

Required Metadata

Current code version

Main text

1. Motivation and significance
   GENESIS [1] and its MPI version PGENESIS are among the longest-maintained compartmental simulators. Models are written as SLI scripts, cell files and channel prototypes, often accumulated over many

*Corresponding author.
Email addresses: `karol@chlasta.pl` (Karol Chlasta), `gmwojcik@umcs.edu.pl` (Grzegorz Marcin Wójcik)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.6.0
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/WarsawIQ/genesis-2.5` (release v2.6.0, archived at `https://doi.org/10.5281/zenodo.23298087`)
C3	Legal Code License	GNU General Public License v2 or later (program); GNU Lesser General Public License v2.1 or later (library portions); see `LICENSE`
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C, CUDA C++, OpenCL C, MPI (PGENESIS), Python and POSIX shell (reproduction and analysis)
C6	Compilation requirements, operating environments & dependencies	Linux x86_64 or aarch64, GCC, GNU make, flex, bison, ncurses. Optional: CUDA 12.x (tested 12.8 on NVIDIA A40 and A100), an OpenCL 1.2 runtime (tested NVIDIA, AMD ROCm 6.3.1, Mesa rusticl 26.1), MPICH or Open MPI for PGENESIS
C7	If available Link to developer documentation/manual	`README.md`, `reproduce/README.md`, `paper/docs/REPLICATION.md`
C8	Support email for questions	karol@chlasta.pl

Table 1: Code metadata (mandatory)

years, and the compute engine is `hsolve`, an implicit Hines solver [10] that compiles a model into a flat program at setup. Until this release every step of that program ran on the CPU.

Relationship to GENESIS 2.4. GENESIS 2.5 is an independently maintained derivative of GENESIS 2.4, released under the same licence as a fork of the project's repository (`genesis-sim/genesis-2.4`, last changed in 2019). It is not an official release of the GENESIS project. The second author has used GENESIS and PGENESIS since his doc-

toral work [18] and met its maintainer, Dave Beeman, at the EU Advanced Course in Computational Neuroscience in Óbidos in 2001; the first author completed his PhD under the second author's supervision in 2023. On 30 September 2026 we offered the work to the original project: the solver fixes as a pull request, and the whole release as its next version. [upstream˙status]

Alternatives, and what they ask of a user. NEURON [11] with its engine CoreNEURON [8] and Arbor [7] run compartmental models on GPUs and are strong choices for a new model. For an existing GENESIS model each means re-expressing it in another description, and establishing that the port computes the same thing is itself experimental work. For the published network used below, it took a transcription of the channels and a comparison of spike trains (Section 3). NEST [12] and Brian 2 [13] simulate point neurons. GENESIS 2.5 offers acceleration without migration: the same scripts on the hardware the user has.

Contributions.

(a) CUDA and OpenCL backends for `hsolve`, selected at run time, in single or double precision, falling back to the CPU for anything they cannot compute.

(b) `hines_tree_eliminate`, a GPU Hines elimination for populations of dendritic trees.

(c) Two fixes to the Hines solver of GENESIS 2.4, which with several cells under one solver computed wrong voltages (backward Euler) or diverged and crashed (Crank–Nicolson), and model construction in linear time; both apply with or without a GPU and are offered to the original project.

(d) A comparison with NEURON, CoreNEURON and Arbor in which every arm is measured in one session, and a reproduction package that regenerates every number in this paper from raw data (Section 3).

2. Software description

(a) Software architecture

The backends sit behind one entry point in `hsolve` (Fig. 1). The SLI interpreter, the element tree, the scheduler and the PGENESIS MPI layer are those of GENESIS 2.4. A user selects a backend with `chanmode` and environment variables. Nothing in a model script changes.

An accepted model runs on one of three paths. *Per step*: the device updates channels and solves each step, and the host delivers synaptic events and spikes between steps. This is the path of any

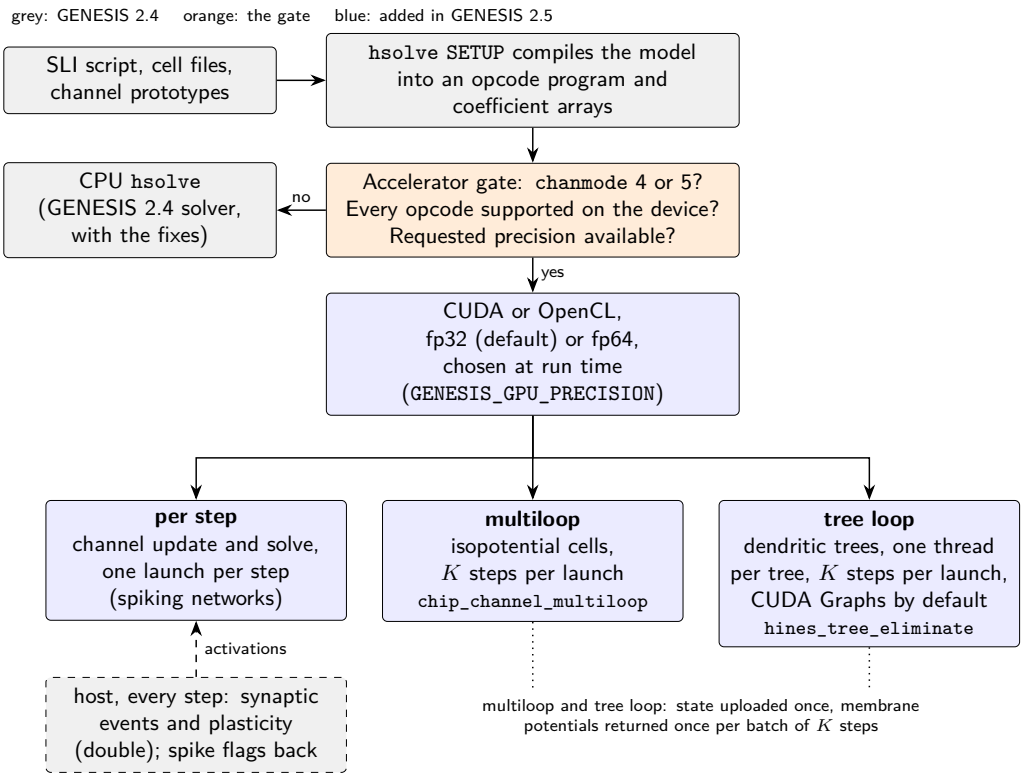


Figure 1: How a model reaches the GPU. `hsolve` compiles the model into an opcode program at `SETUP`. A gate checks the program against what the kernels implement; anything else stays on the CPU. Accepted models run as one of three dispatches: per step, batched over many steps for isopotential cells (`chip_channel_multiloop`), or batched for dendritic trees (`hines_tree_eliminate`). Synaptic events are delivered by the host every step.

network whose cells exchange spikes. A spike emitted in one step therefore reaches its targets in the next; the CPU solver delivers it within the step to the cells after the sender in its loop, so with zero axonal delay the two can differ by one step for some synapses (Section 3, correctness over a long run). *Multiloop*: for isopotential cells without synaptic input, one launch runs K steps. *Tree loop*: for populations of dendritic trees, the channel update and `hines_tree_eliminate` run back to back for K steps. On the two batched paths the state goes to the device once and the membrane potentials come back once per batch, so a run of K steps costs one transfer instead of K.

Design decisions.

i. *The kernels interpret the solver's own program.* Both backends execute the opcode program `hsolve` already builds, so

every model it accepts runs unchanged. GPU-native simulators generate code per mechanism instead. Interpreting costs divergence and a fixed setup time (Section 3), and is why existing models need no porting.

ii. *The tree solve needs no synchronisation.* The elimination program of a population is block-diagonal: each tree touches only its own rows. One thread per tree runs the CPU's elimination with no barrier, once the opcode that joins two trees in the CPU program is split per tree.

iii. *What the kernels cannot compute is refused, not approximated.* The first release accepted models with synaptic, spike, GHK and concentration elements and returned plausible wrong voltages. Such programs are now detected at `SETUP` and computed on the CPU.

iv. *The device owns the state it computes.* With synaptic input the host writes only the slots it owns each step. Uploading the whole state, as the first release did, overwrote gating variables the kernel had just updated.

Execution model. Four properties of the device code decide where it pays off. *Work imbalance.* `hines_tree_eliminate` runs one thread per tree, so the threads of a warp wait for the largest tree among them. A population that mixes trees of 8 and 64 compartments in every warp takes $1.08\times$ as long per step as uniform trees with the same total number of compartments, about what the CPU itself pays for the same mix ($1.09\times$), so the wait within a warp costs little. The same mixed trees grouped by size take $1.27\times$ (E7). Grouping is worse, not better: it puts all the long trees on some of the multiprocessors, which then finish last, while interleaving spreads them across the device. *Divergence.* Compartments with different channel sets follow different opcode streams through the interpreter, so heterogeneous cells cost more than their compartment count suggests. *Transfers.* On the batched paths transfers are fixed per batch, not per step (above). On the per-step path they are paid every step. *Launch overhead.* In the tree loop on the A100 the host spends 6.2 µs per step launching the two kernels, which take 76 µs (Nsight Systems, $N = 10\,000$ trees); the launches overlap the kernels, so the host is not what limits the loop. CUDA Graphs, which replay a recorded sequence of launches, cut the host's part to 0.5 µs per step and recover $3.0 \pm 1.0\%$ of the run time on the A100 and $1.9 \pm 0.5\%$ on the A40 in a run of $K = 50\,000$ steps (E8); at $K = 5000$ the gain is

$4.8 \pm 2.3\%$ on the A100 and, within the noise, $0.7 \pm 1.2\%$ on the A40. They change no result, so they are on by default for the tree loop. On the spiking network they gain nothing within error ($-2.4 \pm 3.9\%$): its cost is the host's event delivery and the synchronisation it forces every step, not the launches. Moving spike delivery to the device is the next step for that workload.

(b) Software functionalities

Precision. Single precision is the default. `GENESIS_GPU_PRECISION=fp64` selects double precision at run time in the same binary, and a device without it refuses with a message and computes on the CPU. In fp64 the GPU reproduces the CPU solver to every printed digit on our verification models; in fp32 it agrees to 2×10^{-7} V over one action potential. Synaptic weights and plasticity are computed by the synapse objects on the host in double precision in every mode. Only the resulting activation reaches the device. What double precision costs on each card, and how the two precisions compare over a 10 s run, are measured in Section 3.

Portability. The OpenCL backend runs the same kernels on any OpenCL 1.2 device. We test it on the NVIDIA A40 and A100 and on the AMD Radeon 890M integrated GPU under two independent implementations, AMD ROCm and Mesa rusticl (E9). The two differ even in what they offer on the same chip: ROCm exposes double precision and rusticl does not, so under rusticl a request for fp64 is refused and the model computes on the CPU. On an integrated GPU that also drives the display, a long tree dispatch trips the driver's watchdog (in our tests from about 22 000 compartments). The backend therefore declines the batched tree dispatch above 20 000 compartments there, says so, and runs the model step by step, unless `GENESIS_OCL_TREE_MAX_NCOMPTS` says otherwise; E9 checks that it does. The same kind of limit stops the longest single-compartment run under rusticl, which uses the graphics queue: the network of $N = 50\,000$ cells batched over 50 000 steps in one dispatch is reset by the driver. Under rusticl such a model has to run one dispatch per step (47.5 s, slower than the laptop's CPU at 32.2 s), while ROCm runs it batched in 5.7 s.

(c) Sample code A model needs no change beyond the solver mode that GENESIS 2.4 already has. An existing script that builds its solver as

```
create hsolve /net/solver
setfield /net/solver path "/net/##[][TYPE=compartment]" \
```

```
    chanmode 4
call /net/solver SETUP
```

runs on the GPU when started with the accelerated binary. Batching and precision are chosen when the run starts, for example

```
GENESIS_CUDA_MULTILOOP=<K> GENESIS_GPU_PRECISION=fp64 \
    nxgenesis -batch model.g
```

The start-up lines name the backend, the precision and the dispatch path in use, and a model the device cannot compute is reported and run on the CPU.

3. Illustrative examples Every number in this section was measured on release v2.6.0 on the UMCS "Lunar" cluster (inf02: Xeon Gold 6342 and NVIDIA A40; inf03: Xeon Platinum 8358 and NVIDIA A100). CPU arms are single-threaded. Every time is wall clock around the whole process, every value a mean over replicates run in interleaved order, every run, CPU arms included, bound to the GPU's NUMA node so that both arms of a speedup use the same socket, and every speedup the ratio of two means, with the arms' relative sample standard deviations combined in quadrature as its uncertainty. On inf02 the A40's socket is the slower one: a CPU arm takes 1.19× as long there as on the other socket, so the A40's speedups would be that much smaller against it; on inf03 the sockets differ by 0.99×. The CPU arms use the same `hsolve` mode as the GPU arms (`chanmode 4`). On the trees `chanmode 3` is faster still by 1.08×, while `chanmode 1`, our CPU reference before this revision, is 4.2× slower with the same voltages.

   (a) Speed on populations of cells Table 2 gives the single-compartment network's speedups; Fig. 2 the dendritic-tree sweep, end to end and step phase. Fig. 3 shows the end-to-end gain growing with run length as the fixed setup cost amortises: the largest speedup is 17.1× on the A40 and 24.0× on the A100, and double precision costs 1.50× and 1.17× the fp32 time on the two cards.

	NVIDIA A40		NVIDIA A100	
N	CUDA	OpenCL	CUDA	OpenCL
500	1.5 ± 0.0	1.5 ± 0.0	1.6 ± 0.1	1.8 ± 0.1
5 000	8.3 ± 0.1	8.3 ± 0.1	8.6 ± 0.3	9.2 ± 0.5
50 000	19.3 ± 0.2	19.3 ± 0.2	20.6 ± 0.2	20.6 ± 0.2

Table 2: Single-compartment network, $K = 50\,000$ steps: speedup of each backend over the CPU solver (E3, 10 replicates).

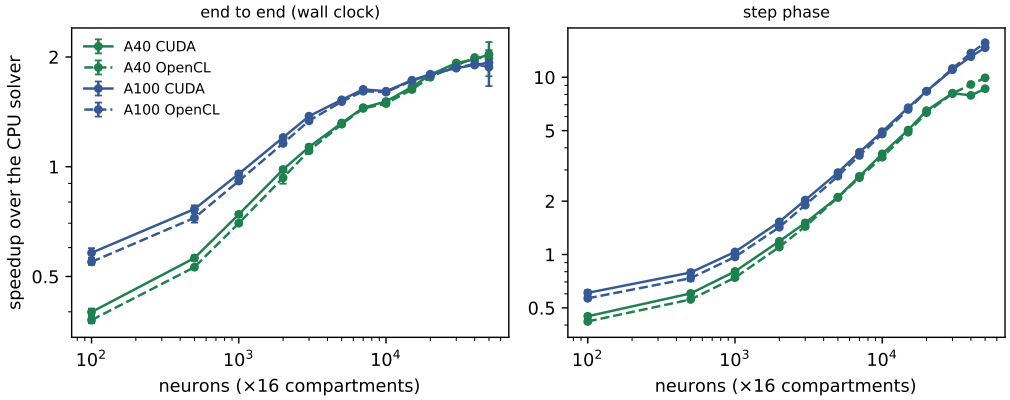


Figure 2: Dendritic trees, $N \times 16$ compartments, 200 steps: speedup over the CPU solver, end to end and step phase, on the A40 and A100 (E3).

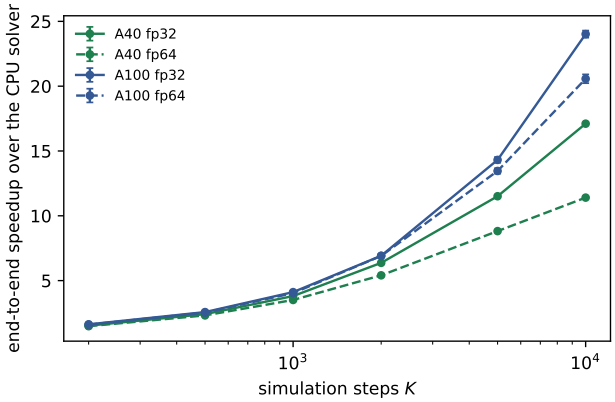


Figure 3: End-to-end speedup against run length, 10 000 × 16 compartments, fp32 and fp64, both cards (E2).

(b) Against other simulators Table 3 lists every arm of the spiking-network comparison, measured in one session on one node in interleaved order, so no ratio in it mixes machines or days. On the dendritic trees, GENESIS and Arbor trade a fixed cost for a marginal one, so in single precision neither is faster in general: on the A100 the two cross at 48 ms of simulated time, on the A40 at 13 ms (Fig. 4). In double precision, which Arbor always uses, they do not cross: GENESIS is slower at every run length we measured, taking 1.10× Arbor's time on the A100 and 1.05× on the A40 at the longest run. The advantage over Arbor is the advantage of single precision. CPU profiles (E6) show where the time goes. On

the CPU, GENESIS is faster than CoreNEURON on both workloads, 2.22× on the network and 2.56× on the trees, because it reads its channel gates from tables, while CoreNEURON evaluates exponentials: the exponential function alone takes 47% of CoreNEURON's time on the network. On the network GENESIS spends 38% of its time on synapses, spike events and the calls that reach them, against CoreNEURON's 12%; that host-side work is what keeps the GPU from helping there. On the trees the channels take 61% of GENESIS's time and the linear solve 21%, against 43% and 17% of CoreNEURON's; most of the remainder is building the model in both, and idle library threads in CoreNEURON.

Simulator	Hardware	Wall time (s)	Relative
GENESIS 2.5, one solver per cell (as published)	CPU	46.9 ± 1.0	1.36
GENESIS 2.5, one solver per layer	CPU	34.6 ± 0.9	1
GENESIS 2.5, one solver per layer, fp32	A100	39.6 ± 0.6	1.14
GENESIS 2.5, one solver per layer, fp64	A100	40.3 ± 0.5	1.17
NEURON 9.0.2, NMODL channels	CPU	100.1 ± 0.6	2.90
NEURON 9.0.2, ChannelBuilder channels	CPU	128.4 ± 1.5	3.71
CoreNEURON 9.0.2	CPU	76.5 ± 0.2	2.21
CoreNEURON 9.0.2	A100	27.1 ± 0.1	0.79
Arbor 0.10.0	A100	150.0 ± 1.8	4.34

Table 3: Vogels–Abbott COBAHH network [15, 16], 4000 cells, 5 s: wall time of every arm in one session on inf03 (E1, 5 replicates, interleaved). Relative: wall time over that of GENESIS with one solver per layer on the CPU; below 1 is faster.

(c) When to use the GPU Table 4 gathers what the measurements above say about where the accelerator pays off and where it does not.

(d) Correctness over a long run The GPU and CPU solvers do not order spikes the same way. On the GPU a spike emitted in one step reaches its targets in the next; the CPU solver delivers it within the step to the cells after the sender in its loop. With this network's zero axonal delay the two schedules differ by one step (50 µs) for some synapses, and the network is chaotic, so it

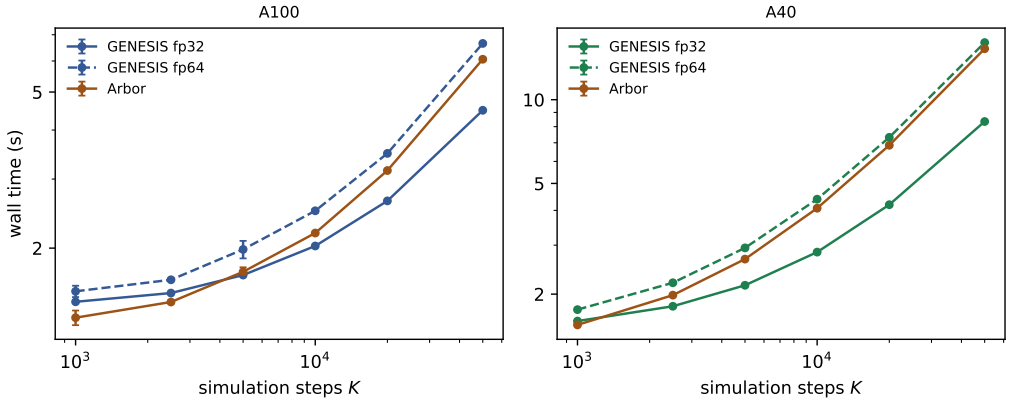


Figure 4: GENESIS and Arbor on the same GPU against run length, GENESIS in fp32 and fp64 (E4).

Workload	GPU?	Why
One detailed morphology	No	one thread per tree; the GPU is slower than the CPU
Population of dendritic trees	Yes, from about 3000 cells	many trees fill the device (Fig. 2)
Isopotential cells without synapses	Yes	21× at $N = 50\,000$ (Table 2)
Spiking network, zero axonal delay	No gain	spikes are exchanged every step, so every step is a dispatch (Table 3)
Long runs	Gain grows with length	the fixed setup cost is amortised (Fig. 3)

Table 4: Where acceleration helps.

amplifies the difference. Over 10 s of the spiking network, twice the length of any comparison above, the GPU's spike train first departs from the CPU's after 2.65 ms in fp64 and 2.65 ms in fp32. The two precisions part at the same step, so the cause is the order, not rounding. The CPU's own order matters as much: run with one solver per cell instead of one per layer, the same network with the same seed departs from itself after 2.35 ms. From then on only statistics can be compared. Against the CPU's 27.2 Hz, the spike count differs by +0.9% in fp64, -0.3% in fp32 and -0.7% for the CPU in the other order; the KS distances between the

interspike-interval distributions are 0.002, 0.003 and 0.006. Four other seeds, other draws of the same network, fire at 24.6–29.2 Hz with KS distances of 0.021–0.049. Each run repeats spike for spike on its own backend. The GPU's spike counts thus differ from the CPU's by about as much as the CPU's differ from themselves under another update order, its interspike intervals are closer to the CPU's than that run's are, and both differences are far smaller than the change from one draw of the network to another.

(e) Reproducing these results Every number in this paper is a row of `reproduce/claims.csv`, naming the data file, the rows and the formula it comes from and the script that measures it; `make_numbers.py` computes the manuscript's values from those files, and a lint lists any number typed by hand. `sh reproduce/run_all.sh` measures them again on the reader's hardware (about 15 minutes for the central claims, 95 for the sweeps; NVIDIA GPU, CUDA 12.x). `--cpu-only` reproduces the hardware-independent results in about two minutes on any Linux machine, and the same path runs in a container (`docker build -t genesis25-cpu . && docker run --rm genesis25-cpu`) with nothing installed. Both run in the repository's CI on every commit. A generated table in the README says which results need which hardware. The comparison simulators and the reference model are built by committed recipes with pinned versions and checksums. Rebuilding them from an empty directory reproduced the published CoreNEURON GPU run to the spike.

4. Impact Two of the release's changes reach every GENESIS user, with or without a GPU. First, the solver fixes: in GENESIS 2.4, putting several cells under one `hsolve` silently gave wrong voltages for every cell under backward Euler (the first cell's were perturbed and the others were never computed), and under Crank–Nicolson the solve diverged or crashed. The configuration is natural for networks, so models may exist whose published results this touched. GENESIS 2.5 computes each cell exactly as if it were alone, shown by a committed test against a single-cell reference, and we have offered the two fixes, with the test, to the original project. **[upstream˙pr˙status]**

   Second, construction. Building a model was quadratic in its size: wall time grew as compartments$^{2.18}$, which made population-scale models impractical regardless of compute power: a 1.7-million-compartment population never finished building. Four scans of the element tree are now constant- or linear-time, construction grows as compartments$^{1.02}$, and the same population builds in 94.9 s (Fig. 5). This holds for models

built element by element, the common case. Copying a whole prototype cell thousands of times under one parent remains quadratic; we say so rather than claim "linear" outright. Together with the tree kernel this moves population studies (parameter sweeps over thousands of morphological neurons, PGENESIS runs partitioned over MPI ranks) from a cluster queue onto the workstation a modeller already has, in the simulator their models are written for. The practical regimes, including where the GPU does not help, are in Table 4.

Beyond GENESIS, two elements of this work transfer. The refusal rule (a backend that cannot compute a model says so and computes on the CPU, rather than approximating) is, we think, the main lesson for anyone adding an accelerator to an existing solver; the first release approximated, and returned plausible wrong voltages. And the reproduction mechanism (a claim map from every published number to its raw data, regenerated into the text, checked in CI) is independent of GENESIS and would serve any computational paper.

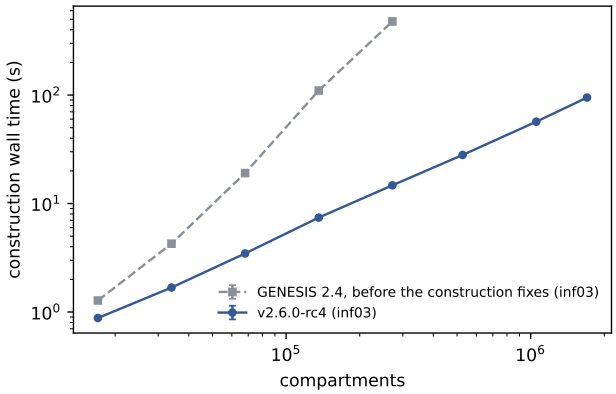


Figure 5: Model construction time against model size on inf03, 3 replicates each: GENESIS 2.4 before the construction fixes (the upstream branch without them, built on the node and timed the same way) and v2.6.0 (E3c). The code from before the fixes grows too fast to measure beyond 16 000 cells.

5. Conclusions GENESIS 2.5 gives a long-established simulator opt-in GPU execution without asking its users to migrate: the same SLI scripts, cell files and MPI workflows run unchanged. The CUDA and OpenCL kernels execute `hsolve`'s own compiled program, in single or double precision chosen at run time. Populations of dendritic trees gain the most, up to 24.0× end to end. Single detailed morphologies and zero-delay spiking networks gain nothing, and the measurements say why: too few trees to fill the device in the first case, host-side event

delivery every step in the second. The next step for that workload is spike delivery on the device. The solver and construction fixes stand on their own and have been offered to the original project. Every number in this paper regenerates from raw data in the repository, the hardware-independent ones on any Linux machine or in a container, so a reader can check what we claim before trusting it on their own models.

CRediT authorship contribution statement

Karol Chlasta: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review and editing. **Grzegorz Marcin Wójcik**: Resources, Supervision, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic's Claude to draft and edit the manuscript and to assist with the software and its benchmarking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Acknowledgements

The authors thank the Maria Curie-Skłodowska University (UMCS) in Lublin and the LubMAN UMCS computing centre for access to the "Lunar" cluster [19], and Karol S. Karpowicz and Prof. Przemysław Stpiczyński for provisioning and support of the compute environment, and WarsawIQ for the AMD Radeon 890M on which the OpenCL backend was developed.

References

[1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System*. Springer, 1998.

[2] GENESIS 3 Development Plans. `http://www.genesis-sim.org/GENESIS/G3/G3-dev-plans.html`

[3] Cornelis H, Coop AD, Rodriguez AL, Beeman D, Bower JM. GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* 2011, 12(Suppl 1):P15.

[4] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88.

[5] Cornelis H, Lu H, Esquivel A, Bower JM. Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* 2007, 8(Suppl 2):P3.

[6] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33.

[7] Akar NA, Cumming B, Karakasis V, Küsters A, Klijn W, Peyser A, Yates S. Arbor – A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In: *27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019, pp. 274–282. doi:10.1109/EMPDP.2019.8671560.

[8] Kumbhar P, Hines M, Fouriaux J, Ovcharenko A, King J, Delalondre F, Schürmann F. CoreNEURON: An Optimized Compute Engine for the NEURON Simulator. *Frontiers in Neuroinformatics* 2019, 13:63. doi:10.3389/fninf.2019.00063.

[9] Markram H, Muller E, Ramaswamy S, et al. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 2015, 163(2):456–492. doi:10.1016/j.cell.2015.09.029.

[10] Hines M. Efficient computation of branched nerve equations. *International Journal of Bio-Medical Computing* 1984, 15(1):69–76. doi:10.1016/0020-7101(84)90008-4.

[11] Hines ML, Carnevale NT. The NEURON Simulation Environment. *Neural Computation* 1997, 9(6):1179–1209. doi:10.1162/neco.1997.9.6.1179.

[12] Gewaltig M-O, Diesmann M. NEST (NEural Simulation Tool). *Scholarpedia* 2007, 2(4):1430. doi:10.4249/scholarpedia.1430.

[13] Stimberg M, Brette R, Goodman DFM. Brian 2, an intuitive and efficient neural simulator. *eLife* 2019, 8:e47314. doi:10.7554/eLife.47314.

[14] Yavuz E, Turner J, Nowotny T. GeNN: a code generation framework for accelerated brain simulations. *Scientific Reports* 2016, 6:18854. doi:10.1038/srep18854.

[15] Vogels TP, Abbott LF. Signal Propagation and Logic Gating in Networks of Integrate-and-Fire Neurons. *Journal of Neuroscience* 2005, 25(46):10786–10795. doi:10.1523/JNEUROSCI.3508-05.2005.

[16] Brette R, Rudolph M, Carnevale T, Hines M, Beeman D, Bower JM, et al. Simulation of networks of spiking neurons: A review of tools and strategies. *Journal of Computational Neuroscience* 2007, 23(3):349–398. doi:10.1007/s10827-007-0038-6.

[17] Chlasta K, Sochaczewski P, Wójcik GM, Krejtz I. Neural simulation pipeline: Enabling container-based simulations on-premise and in public clouds. *Frontiers in Neuroinformatics* 2023, 17:1122470. doi:10.3389/fninf.2023.1122470.

[18] G.M. Wójcik, W.A. Kamiński. Liquid state machine built of Hodgkin–Huxley neurons and pattern recognition. *Neurocomputing* 58–60 (2004) 245–251.

[19] Maria Curie-Skłodowska University (UMCS), Instytut Matematyki. Superkomputer LUNAR uruchomiony w Instytucie Matematyki UMCS. `https://www.umcs.pl/pl/aktualnosci,57,superkomputer-lunar-uruchomiony-w-instytucie-matematyki-umcs,27879.chtm`, 2015.

Current executable software version

Nr.	(Executable) software metadata description	Please fill in this column
S1	Current software version	v2.6.0
S2	Permanent link to executables of this version	`https://github.com/WarsawIQ/genesis-2.5/releases/tag/v2.6.0`
S3	Legal Software License	GNU General Public License v2 or later
S4	Computing platforms/Operating Systems	Linux x86_64; tested on AlmaLinux 8.10 with CUDA 12.8 (NVIDIA A40, A100) and on Ubuntu 24.04 (TUXEDO OS) with ROCm 6.3 and Mesa rusticl (AMD Radeon 890M)
S5	Installation requirements & dependencies	glibc and ncurses; for the GPU binaries an NVIDIA driver for CUDA 12, or an OpenCL 1.2 runtime
S6	If available, link to user manual	`README.md` in the repository
S7	Support email for questions	karol@chlasta.pl

Table 5: Software metadata (optional)